Credential and Access Management Automation with Claude: Research and Content Plan

Cybersecurity · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> Use case 7 from 'Some Use Cases. Yusuf.txt': 'Credential and Access Management Automation: Creating credentials access and raise ticket with respective channel on behalf of new joiner.' Context from the same list: load balancer platforms F5, NetScaler, Radware and Anaya LB (use case 1), Cisco ACI (use case 10), Terraform and GitHub (use case 11), ServiceNow (SNOW, use cases 5 and 6), and the aim 'to do automation using Claude for all these content'.

Joiner access is a multi-queue, multi-system process. A network engineer typically needs ServiceNow requests, local accounts on several load balancers and the ACI fabric, and repository access, each owned by a different team with its own role model: F5 partition roles [14], NetScaler command policies [18], APIC security domains and roles [20] and GitHub team roles [28]. Delays and over-granting both come from people having to remember the right role in each system.

The controls are well defined. NIST SP 800-53 AC-2 asks for approval before account creation, defined criteria for role membership, periodic review and alignment with termination and transfer processes [38], and least privilege means the minimum authorizations needed [6]. These map directly onto a versioned entitlement matrix, approval gates and a leaver workflow.

Claude is useful for reading requests, planning and drafting tickets and explanations, but credential workflows are exactly where OWASP's warnings about sensitive information disclosure and excessive agency apply [7][46]. The course teaches a pattern that keeps the model away from passwords and write paths: code resolves entitlements and calls APIs, people approve, and every step is logged [37].

2. How the outline was shaped

Decision	Rationale
Scope the client's one-line use case to network operations joiners across the client's own platforms.	The client's use case list names F5, NetScaler, Radware, Anaya LB, Cisco ACI, ServiceNow, Terraform and GitHub; covering those systems makes the course directly usable by the client's teams.
Make the entitlement matrix, not the model, the source of authority.	Least privilege [6] and AC-2 role criteria [38] need a reviewable artifact; OWASP recommends authorization outside the LLM [46].
Teach the ServiceNow Table API as the core and the Service Catalog API as the preferred path where catalog items exist.	The outline names sc_request and sc_req_item via the Table API [10]; order_now returns a request through the catalog's own fulfillment [1], which many instances depend on.
Use dry-run, gated apply and verify as the single adapter contract across all devices.	It makes every platform reviewable in the same way and isolates platform defaults such as F5's admin default role [13] and NetScaler's deny-all without a policy [18].
Treat Radware Cyber Controller and Anaya LB as manual-ticket paths unless an API is confirmed in the lab.	No public API reference for local user management on these platforms was found during research; the Alteon REST API is documented but its user objects must be confirmed per version [25][26].
Deliver credentials through Vault response wrapping.	Single-use unwrap gives a concrete one-time channel the model never touches, with tamper signals [35].

3. Audience and prerequisites

Role	What they need	Starting point
Network operations and load balancer engineer	Automate local account creation and removal on F5, NetScaler, Radware and ACI safely.	Strong device CLI knowledge, some REST and Python.
Identity and access administrator	Encode role models, approvals and reviews in an auditable workflow.	Strong IAM process knowledge, limited device APIs.
ServiceNow process owner	Design request, item and approval structures that automation can drive idempotently.	Strong ServiceNow configuration, limited Python.
Platform or DevOps engineer	Own the Terraform and GitHub Actions access-as-code pipeline.	Strong Git, Terraform and CI, limited network devices.

Prerequisites from the outline

- Basic Python and REST API concepts

- Working knowledge of at least one load balancer or network controller (F5, NetScaler, Radware or Cisco ACI)
- Familiarity with ServiceNow request and ticket workflows
- Git basics; familiarity with an AI coding assistant is helpful, not required

Checks before enrolment

- Can you call a REST API with Python requests or httpx and read a JSON response?
- Have you created a local user on at least one of F5, NetScaler, Radware or APIC?
- Can you describe how a ServiceNow request, requested item and approval relate?
- Can you create a branch, commit and open a pull request in Git?
- Do you know why passwords must not appear in tickets or logs?
- Can you install Python 3.11+, Terraform and Claude Code on your laptop?

Course outcomes

- Model a new-joiner access request as a typed record and a versioned role-to-entitlement matrix that Claude plans against
- Set up Claude Code, the Claude Agent SDK and MCP servers so the agent can read ServiceNow and identity data as tools and return structured JSON access plans
- Raise and route ServiceNow requests and requested items per system through the Table API and Flow Designer on behalf of the joiner
- Provision least-privilege local accounts on F5 (iControl REST and tmsh), Citrix NetScaler (NITRO API), Radware Alteon / Cyber Controller, Cisco APIC and other load balancers (Anaya LB) through deterministic, dry-run-first adapters
- Codify GitHub team and repository access as Terraform in a pull request, planned by GitHub Actions and applied only after review
- Enforce guardrails, human approval, one-time credential delivery and a complete audit log, and run the same workflow in reverse for leavers

4. Lab environment

Every lab runs against sandboxes: a ServiceNow PDI per learner, shared virtual F5 BIG-IP VE and NetScaler VPX appliances, a lab Radware Alteon VA where the client can supply one (otherwise a mock adapter), the Cisco DevNet ACI simulator, a throwaway GitHub organization and a dev-mode HashiCorp Vault. Learners run Claude Code and the Agent SDK locally.

Components

Component	Version or edition	Purpose	How to obtain	Notes
ServiceNow Personal Developer Instance	Current release offered by the Developer Program	Requests, requested items, Flow Designer approvals, MCP data source [10][12]	Register on the ServiceNow Developer Program and request a PDI	Instances unused for ten days return to the pool and are reset; have learners log in the day before and keep setup scripts to rebuild [11].
F5 BIG-IP Virtual Edition	A current BIG-IP VE release (the trial covers 12.1.x and later) on KVM or ESXi	Local users via iControl REST and tmsh [13] [14]	30-day trial keys from MyF5 > Trials [15]	Trial is limited to 1 Mbps throughput, which does not matter for management API labs; needs 2 GB RAM per vCPU [15]. Request keys close to the course date because the trial lasts 30 days.
NetScaler VPX Express	NetScaler 13.1 or 14.1 VPX	System users and command policies via NITRO [16][17]	Download VPX Express (free, no license file) from NetScaler [19]	Maximum five Express downloads per customer and no support cases [19]; one appliance can be shared if each learner uses a name prefix.
Radware Alteon VA	A build at or above the SDK minimums (31.0.9.0 / 32.2.1.0); confirm REST support on the chosen build	Local users and manual fallback [25] [26][27]	Through the client's Radware account team or partner; no public trial page could be verified during research	If unavailable, the lab uses a mock Alteon adapter with recorded responses and the CLI fallback ticket.
Cisco ACI Simulator (DevNet)	As provided by DevNet sandbox	APIC local users, domains and roles [20] [21]	Reservable ACI Simulator sandbox (up to five days) or the shared always-on sandbox [22]	The always-on instance is shared; use unique user names and clean up. Prefer a reservation for the course week.
GitHub organization	github.com Free or Team	Team membership, Terraform provider target, Actions [28][32]	Instructor creates a throwaway organization and learner test accounts	Invitation rate limits apply to new free organizations [29].
Terraform CLI and	Terraform 1.11+ and the current	Access as code [30][31]	HashiCorp install page; provider	1.11+ is needed for write-only arguments [34]; state

GenNoor Tech
For more info, https://gennoor.com

github provider	integrations/github provider release		from the Terraform Registry	in a remote backend.
HashiCorp Vault	Vault 1.x in dev mode (lab only)	One-time credential delivery via response wrapping [35]	Vault binary or container; vault server -dev	Dev mode is in-memory and insecure by design; never use it beyond the lab.
Claude Code and Claude Agent SDK (Python)	Current releases	Assisted build, orchestrator, MCP tools [39] [40][41]	Install per Anthropic documentation; course organization account	Ship .claude/settings.json with deny rules and hooks in the lab repo [42][43].
Python	3.11 or 3.12	Adapters, resolver, FastAPI callback, Pydantic schemas	python.org	Pinned requirements: pydantic v2, httpx, fastapi, pyyaml, hvac, claude-agent-sdk, anthropic.

Fixtures and sample data

Fixture	Contents	Used in
fixtures/joiner_E10422.json and two invalid records	Valid joiner, malformed record, out-of-matrix request	Joiner intake and entitlement resolution; Capstone: one joiner, every system, fully audited
access/matrix.yaml and schema	Three roles with per-system entitlements, owners, rule ids	The new-joiner access workflow and the automation baseline; all later modules
runbook.md	Team's as-is manual runbook with inconsistencies	The new-joiner access workflow and the automation baseline
servicenow/setup/	Scripts to create groups, the idempotency field and the approval flow on a PDI	Raising tickets in the right channel with ServiceNow
inventory.yaml	lb-f5-01, lb-f5-02, lb-ns-01, lb-rdw-01, apic-01, anaya-lb-01 with platform keys	Device accounts on F5 and Citrix NetScaler; Radware, Cisco ACI and other load balancers (Anaya LB)
drift seed script	Creates a stale admin account and an unknown user for access review	Verification, hand-over and reversal for leavers and movers

Lab safety rules

- Use only lab devices, PDIs and the throwaway GitHub organization; never point adapters at production.
- Adapter service accounts get the minimum role that can manage users (for example User Manager on F5) [14].
- No password, token or session id in prompts, tickets, logs or Git [7][37].
- Every create, modify and delete requires an approval reference re-read at apply time.
- Tear down all lab users, tokens, PDI data and the GitHub organization after the course.

5. Module-by-module content plan

Day 1

Module 1: The new-joiner access workflow and the automation baseline

Goal. Map the real joiner flow and turn the team's tribal knowledge into a versioned, owned role-to-entitlement matrix.

Outline topics.

- Walking the real flow: HR trigger, manager request, per-system tickets, local device accounts, repository access, hand-over to the joiner
- Which systems network operations teams actually touch: F5, NetScaler, Radware, Cisco ACI, ServiceNow, GitHub and Terraform, plus other load balancers (Anaya LB)
- The teaching pattern used in every build: Claude plans, drafts and explains; deterministic code executes; a human approves; everything is logged
- Mapping tribal knowledge into a role-to-entitlement matrix with owners, scopes and expiry

Learning objectives.

- Learners can draw the joiner flow from HR trigger to hand-over and mark every hand-off and queue.
- Learners can list the systems a network operations joiner needs and who fulfills each.
- Learners can explain the course pattern: Claude plans, drafts and explains; deterministic code executes; a human approves; everything is logged.
- Learners can write an entitlement matrix in YAML with roles, per-system entitlements, owners, scopes and expiry.
- Learners can relate the workflow to account-management controls such as approval before creation and periodic review.

Walking the real flow

- Trigger: an HR or identity record (for example a new user in Microsoft Entra ID, created with POST /users [3]) or an HR export.

Gennoor Tech TRAIN INNOVATE BUILD For more info - https://gennoor.com

- Manager request, then one ticket per system, local device accounts, repository access and a hand-over message.
- Count the queues: in the promo scenario five tickets sit in five queues; learners map their own team's version.
- Note every place a person must 'remember the right role'; those are the matrix rows.

Systems a network operations joiner touches

- ServiceNow for requests and approvals; F5 BIG-IP, NetScaler, Radware Alteon and other load balancers (Anaya LB) for device accounts; Cisco ACI APIC for fabric access; GitHub and Terraform for code.
- Teams on Jira Service Management can swap the ticketing adapter: POST /rest/servicedeskapi/request with serviceDeskId, requestTypeId and requestFieldValues [2].
- Each system has its own role model: F5 partition roles [14], NetScaler command policies [18], APIC security domains and roles [20], GitHub team roles [28].
- Anaya LB is named in the client input; no public API documentation was found during research, so it is treated through the adapter interface and a manual ticket fallback.

The teaching pattern used in every build

- Claude plans and drafts (access plans, ticket text, explanations); Python adapters make every API call.
- A person approves anything that creates, changes or removes a credential; AC-2 requires approvals by defined personnel for requests to create accounts [38].
- Authorization lives outside the model: OWASP calls this complete mediation [46].
- Every step is logged with who, what, when and where [37].

From tribal knowledge to an entitlement matrix

- Least privilege means the minimum resources and authorizations needed for the function [6]; the matrix is where that minimum is written down.
- Row shape: role, system, entitlement (e.g. f5: partition Common, role operator), owner, approver group, expiry or review date, rule id.
- Version the matrix in Git (matrix v1.4 in the promo) and change it only by pull request.
- AC-2 also asks for defined criteria for group and role membership and periodic account review [38]; the matrix supplies both.

Instructor demo.

1. Show a sample joiner record (fixtures/joiner_E10422.json) and the team's current runbook as a wiki page.
2. Draw the current flow on a whiteboard and mark each queue and each manual role decision.
3. Ask Claude Code to propose a first entitlement matrix from the runbook text, then point out where it guessed.
4. Correct the matrix by hand, add owners and rule ids, and commit it as access/matrix.yaml v1.0.

5. Show how a matrix row maps to the exact role names on F5 (operator) and NetScaler (operator command policy) [14][18].

Hands-on lab. Walk through a sample joiner record and the team's current runbook, then draft the entitlement matrix for three roles as a versioned YAML file.

1. Read the joiner record and runbook in fixtures/.
2. Draw the as-is flow and list every system, queue and approver.
3. Choose three roles: lb-operator, lb-engineer and dc-network-ops.
4. For each role and system, write the minimum entitlement using the platform's real role names.
5. Add owner, approver group, expiry or review date and a rule id per row.
6. Write a JSON Schema for the matrix and validate the YAML against it.
7. Commit as matrix v1.0 with a short CHANGELOG entry.

Lab success criteria.

- Every role-system pair has an explicit entitlement or an explicit 'none'.
- No row grants an administrator role without a named owner and justification.
- The matrix validates against its schema.
- The as-is flow diagram names every queue and approver.

Deliverable. Documented workflow and entitlement matrix v1.

Common pitfalls.

- Copying 'whatever the last joiner got' into the matrix, which bakes in privilege creep.
- Using display names instead of the platform's actual role identifiers.
- Leaving owner or expiry blank, which makes periodic review impossible.
- Letting the model fill gaps in the matrix without marking them for review.

Knowledge check.

1. In the course pattern, who makes the API call that creates an account? Answer: Deterministic Python adapter code, after a human approval; Claude only plans and drafts.
2. What does least privilege mean for a matrix row? Answer: The row grants only the minimum resources and authorizations the role needs [6].
3. Which AC-2 items does the matrix help satisfy? Answer: Criteria for role membership, approval before account creation and periodic account review [38].

Slides and assets to prepare.

- As-is and to-be joiner flow diagrams.
- Matrix YAML template and JSON Schema.
- Role-name crosswalk: F5 roles, NetScaler command policies, APIC roles and domains, GitHub team roles.
- Sample runbook with deliberate inconsistencies for the lab.

Sources for this module. 2, 3, 6, 14, 18, 20, 28, 37, 38, 46

Module 2: Setting up Claude Code, the Agent SDK and MCP for the access workflow

Goal. Stand up the project with safe Claude Code settings, an Agent SDK orchestrator with narrow tools, read-only MCP access to ServiceNow and the identity source, and a validated access-plan schema.

Outline topics.

- Claude Code project layout, CLAUDE.md conventions and permission settings for a credential workflow
- Claude Agent SDK orchestrator: tool definitions for read_joiner, lookup_entitlements, draft_ticket and plan_account
- MCP servers that expose ServiceNow records and the HR or identity source as read-only tools
- Structured JSON outputs validated with Pydantic so every access plan is an object, never free text

Learning objectives.

- Learners can write CLAUDE.md conventions and permission rules that keep secrets out of the assistant's reach.
- Learners can define Agent SDK tools read_joiner, lookup_entitlements, draft_ticket and plan_account with narrow schemas.
- Learners can connect an MCP server for ServiceNow records in read-only mode and explain its trust implications.
- Learners can get a Pydantic-validated access plan from the agent and reject invalid output.

Claude Code project layout and settings

- Put conventions in ./CLAUDE.md (run /init to start) and personal sandbox URLs in CLAUDE.local.md, which stays out of Git [44].
- Add deny rules such as Read(./.env) and Read(./secrets/**) in .claude/settings.json; rules are evaluated deny, then ask, then allow [42].
- Bash permission rules match command text and are not a security boundary, so secrets must not be reachable from the workstation session at all [42].
- Anthropic does not audit MCP servers; use your own or trusted ones and review commands before approval [9].

The Agent SDK orchestrator

- Define tools with @tool(name, description, schema) and wrap them in create_sdk_mcp_server(name='access', version='1.0.0', tools=[...]) [39].
- Pre-approve only read tools in allowed_tools, e.g. mcp__access__read_joiner and mcp__access__lookup_entitlements [39].

Genmoor Tech
For more info - https://genmoor.com

- Use permission_mode 'dontAsk' for the headless planner so anything not pre-approved is denied; never use bypassPermissions, which ignores allowed_tools [40].
- plan_account returns a proposed plan object only; it has no device credentials and no write path, following OWASP's 'minimize extension functionality' [46].
- Mark read tools readOnlyHint: true, remembering annotations are metadata, not enforcement [39].

MCP servers for ServiceNow and identity data

- Add a project-scoped server with claude mcp add --transport stdio --scope project servicenow -- python -m mcp_servicenow; it lands in .mcp.json [41].
- Pass instance URL and credentials with ${VAR} expansion, not literal values in .mcp.json [41].
- The ServiceNow integration user behind the MCP server gets read roles only on the tables it needs; the Table API enforces the calling user's roles [10].
- MCP servers must not pass through tokens not issued to them, and should start with minimal scopes [8].
- Project MCP servers load without a prompt in Agent SDK runs, so review .mcp.json in code review [41].

Structured access plans

- AccessPlan v2: joiner_id, matrix_version, items[{system, target, entitlement, rule_id, action}], rejected[{entitlement, reason, rule_id}].
- Request it with output_config.format of type json_schema, or client.messages.parse() with the Pydantic model [45].
- Keep the schema within supported features: additionalProperties false, no numeric minimum/maximum, no recursion [45].
- Validate again in code with Pydantic; an invalid plan is a failed run, never 'best effort'.

Instructor demo.

1. Open the project skeleton and walk through CLAUDE.md, .claude/settings.json deny rules and .mcp.json.
2. Ask Claude Code to 'read the .env file' and show the deny rule blocking it [42].
3. Register the ServiceNow MCP server against the PDI and list sys_user and sc_req_item records through it.
4. Run the orchestrator for E10422 and show the tool calls in order (read_joiner, lookup_entitlements, plan_account) [39].
5. Show the returned access_plan.json validating against AccessPlan v2, then break the schema and show the failure.

Hands-on lab. Stand up the project, connect the ServiceNow MCP server to a developer instance, and have the agent return a validated access plan for one joiner.

1. Request or wake your ServiceNow PDI and create an integration user with read-only roles on the needed tables [11].
2. Clone the lab repository and review CLAUDE.md and .claude/settings.json; add deny rules for .env and secrets/.
3. Configure the ServiceNow MCP server in .mcp.json with ${SN_INSTANCE} and ${SN_USER} variables [41].
4. Implement read_joiner and lookup_entitlements as Agent SDK tools with JSON Schema inputs [39].
5. Write the AccessPlan Pydantic model and use it for structured output [45].
6. Run the orchestrator in dontAsk mode with only read tools pre-approved [40].
7. Save access_plan.json and run the validation tests.

Lab success criteria.

- The agent returns a plan that validates against AccessPlan v2.
- No tool available to the agent can write to ServiceNow or any device.
- Claude Code cannot read .env or secrets/ in the project.
- The MCP server uses an integration user with read-only roles.

Deliverable. Working agent skeleton with MCP tools and an access-plan schema.

Common pitfalls.

- Putting the ServiceNow password directly in .mcp.json and committing it.
- Running the orchestrator in bypassPermissions to 'save time'.
- Giving the agent a generic http_request tool instead of narrow tools.
- Treating CLAUDE.md instructions as a security control.

Knowledge check.

1. Why use dontAsk mode for the headless planner? Answer: Any call that is not pre-approved is denied instead of prompting, so the tool surface is fixed [40].
2. What does allowed_tools do in bypassPermissions mode? Answer: Nothing to restrict; every tool is approved, so it is not used in this course [40].
3. Where should the ServiceNow credentials for the MCP server come from? Answer: Environment variables referenced with ${VAR} in .mcp.json, never literal values in the file [41].

Slides and assets to prepare.

- Project layout diagram (CLAUDE.md, .claude/, .mcp.json, orchestrator/, adapters/, schemas/).
- Annotated .claude/settings.json and .mcp.json.
- AccessPlan v2 schema and example.
- Permission evaluation order slide (hooks, deny, ask, mode, allow, callback).

Sources for this module. 8, 9, 10, 11, 39, 40, 41, 42, 44, 45, 46

Gemnoor Tech TRAINING INNOVATE BUILD For more info - https://gelnoor.com

Module 3: Joiner intake and entitlement resolution

Goal. Validate the joiner record, resolve role to per-system entitlements from the matrix, reject anything incomplete or out of matrix with a reason, and diff against existing access.

Outline topics.

- Typed joiner record: employee ID, team, role, location, start date, manager, requested systems and justification
- Resolving role to entitlements per system: F5 partition and role, NetScaler command policy, APIC security domain and role, GitHub teams
- Rejecting incomplete or out-of-matrix requests with an explicit reason instead of a guess
- Diffing the proposed plan against what the joiner already has

Learning objectives.

- Learners can implement a typed joiner record and reject malformed input with field-level errors.
- Learners can resolve entitlements deterministically from the matrix for each target system.
- Learners can produce explicit rejections with rule ids for out-of-matrix requests.
- Learners can diff the plan against current access read from each system.

The typed joiner record

- Fields: employee_id, name, team, role, location, start_date, manager_id, requested_systems, justification.
- Strict validation: unknown fields rejected, start_date not in the past, manager must exist in the identity source.
- Model-proposed values (for example parsing a free-text request email) are marked as proposed and re-validated in code [45].
- Treat request text as untrusted; it can contain instructions aimed at the model [7].

Resolving entitlements per system

- F5: partition and role, e.g. Common/operator; if the role is omitted in iControl REST it defaults to admin, so the resolver always sets it [13].
- NetScaler: a command policy such as operator or read-only; a user with no bound policy is denied all configuration commands [18].
- APIC: security domain plus role and privType, e.g. domain netops, role ops, writePriv [20].
- GitHub: team and team role (member or maintainer) [28].

Rejecting with a reason

- Out-of-matrix request: 'f5:admin not granted to lb-operator (matrix rule R-12)', as in the promo.
- Incomplete record: list each missing or invalid field.
- Rejections are part of the plan output, not exceptions hidden in logs.

- The resolver is code; Claude explains rejections in plain language for the requester but cannot overturn them [46].

Diffing against existing access

- Read current state: GET /mgmt/tm/auth/user on F5 [13], GET /nitro/v1/config/systemuser on NetScaler [16], the user subtree under uni/userext on APIC [20], team memberships on GitHub [28].
- Plan actions per item: create, modify, none, or flag (account exists with more privilege than the matrix allows).
- Flags become review tickets, not silent downgrades.
- Store the diff in the plan so approvers see exactly what changes.

Instructor demo.

1. Feed the valid E10422 record and show the resolved plan with seven items.
2. Feed a record missing manager_id and show field-level rejection.
3. Request f5:admin for an lb-operator and show the R-12 rejection.
4. Run the diff against a lab F5 where a stale account with admin role exists and show the flag item.
5. Ask Claude to explain the rejection for the requester and show it cannot change the plan.

Hands-on lab. Implement the intake validator and entitlement resolver with Claude Code, then feed it a malformed record and an out-of-matrix request and inspect the rejections.

1. Ask Claude Code to draft the JoinerRecord Pydantic model from the field list; review it before accepting.
2. Implement validation rules (dates, manager existence, allowed systems).
3. Implement the resolver that reads matrix.yaml and returns per-system entitlements with rule ids.
4. Always set explicit roles in F5 and NetScaler entitlements.
5. Implement read-only current-state readers using fixtures first, then the lab devices.
6. Implement the diff and add its result to the access plan.
7. Run the three provided records (valid, malformed, out-of-matrix) and save outputs.

Lab success criteria.

- The malformed record is rejected with every bad field named.
- The out-of-matrix request is rejected with the matrix rule id.
- Every plan item has an explicit role or policy, never a platform default.
- The diff shows create, modify, none or flag for each item.

Deliverable. Validated joiner record and per-system entitlement plan.

Common pitfalls.

- Relying on platform defaults for roles (F5's default is admin).

Gemnoor Tech TRAINING BUILD For more info - https://gemnoor.com

- Silently fixing bad input instead of rejecting it.
- Letting the model resolve entitlements from the matrix text instead of code.
- Ignoring existing over-privileged accounts during the diff.

Knowledge check.

1. What role does an F5 user get if the partition-access role is omitted? Answer: admin, which is the documented default, so the adapter must always set it [13].
2. A NetScaler user is created but no command policy is bound. What can they do? Answer: No configuration commands; the default is DENY-ALL [18].
3. What should happen when the diff finds an existing account with more privilege than the matrix allows? Answer: Flag it for review with a ticket; do not silently change it.

Slides and assets to prepare.

- Joiner record schema card.
- Three fixture records (valid, malformed, out-of-matrix).
- Resolver flow diagram.
- Example plan with diff and rejections.

Sources for this module. 7, 13, 16, 18, 20, 28, 45, 46

Day 2

Module 4: Raising tickets in the right channel with ServiceNow

Goal. Raise the request and one requested item per system, route each to its fulfillment group, gate provisioning on Flow Designer approval and never create duplicates.

Outline topics.

- ServiceNow Table API: creating sc_request and one sc_req_item per system, setting assignment_group, requested_for and short_description
- Letting the agent draft the ticket description and justification from the plan while the Python client owns the call
- Flow Designer for manager approval, state transitions and the callback that unblocks provisioning
- Idempotency: never raise a duplicate ticket for the same joiner and system

Learning objectives.

- Learners can create and query ServiceNow records with the Table API and explain when to use the Service Catalog API instead.
- Learners can set requested_for, assignment_group and short_description per item from the plan.
- Learners can design a Flow Designer approval and a callback that unblocks provisioning.
- Learners can make ticket creation idempotent.

Table API essentials

- POST /api/now/table/sc_request creates one record per call and returns 201 with a Location header [10].
- Use sysparm_fields=number,sys_id to keep responses small and sysparm_input_display_value only when you deliberately send display values [10].
- GET with sysparm_query (e.g. requested_for=<sys_id>^active=true) finds existing records; ^ means AND [10].
- The integration user needs roles for each table; the API enforces them [10].

Catalog API versus direct records

- If your organization has a catalog item per system, POST /api/sn_sc/servicecatalog/items/{sys_id}/order_now with sysparm_quantity and sysparm_requested_for returns the request number and runs the item's normal fulfillment [1].
- Direct Table API inserts into sc_request and sc_req_item match the outline and suit teams without catalog items, but check with the platform owner which flows they trigger on your instance.
- Teams on Jira Service Management use POST /rest/servicedeskapi/request with requestTypeId and requestFieldValues instead [2].
- Either way, the Python client owns the call; the agent only drafts description and justification text.

Routing and drafting

- Per item: requested_for (joiner sys_id), assignment_group (lb-fulfilment, dc-network, platform-eng), short_description from a template.
- Claude drafts the description and justification from the plan; code checks the draft names only the systems and roles in that plan item.
- Never include credentials or sensitive personal data in ticket text [7].
- The request links back to the plan id and matrix version for audit.

Flow Designer approval and the callback

- Ask for Approval with a manager rule (anyone approves) returns an approval state of approved, rejected or cancelled; follow it with If/Else logic [12].
- Approval responses are stored in sysapproval_approver, which the audit step reads [12].
- Set a due date with an explicit outcome (reject) so requests do not wait forever [12].
- On approval the flow calls back (REST step or outbound event) to the orchestrator, which re-reads the record state before provisioning rather than trusting the payload.

Idempotency

- Idempotency key = joiner_id + system + plan item hash, stored in a dedicated field on the item.
- Before POST, GET with sysparm_query on that field; if a record exists, reuse it [10].

- Retries after timeouts must query first, because a POST may have succeeded even if the response was lost.
- Log every create and reuse decision with the key.

Instructor demo.

1. Run access-agent tickets --plan access_plan.json --dry-run and show the request bodies and routing table.
2. Create the sc_request with the Table API and show the 201 response and Location header [10].
3. Create the five requested items, each routed to its group.
4. Show Flow Designer's Ask for Approval waiting on the manager and approve it in the PDI [12].
5. Show the callback reaching the orchestrator and the orchestrator re-reading the approval state.
6. Re-run the same command and show zero new records created.

Hands-on lab. Raise the full set of tickets for one joiner against the developer instance, route each to its fulfilment group and watch Flow Designer approval gate the next step.

1. Create assignment groups and an idempotency field on sc_req_item in your PDI (instructions provided).
2. Implement the ServiceNow client with create, get-by-query and update methods [10].
3. Implement the ticket orchestrator that maps plan items to requested items and groups.
4. Add the Claude drafting step for description and justification, with a check that the text matches the plan item.
5. Build the Flow Designer flow with Ask for Approval, a due date and If/Else on the outcome [12].
6. Add the callback endpoint (FastAPI) and the state re-read.
7. Run dry-run, then live; approve in the PDI; confirm provisioning is unblocked.
8. Run again to prove no duplicates.

Lab success criteria.

- One request and one item per system, each in the right group.
- Provisioning does not start until the approval state is approved.
- A second run creates no new records.
- Ticket text contains no credentials.

Deliverable. Ticket orchestrator with per-channel routing and approval callback.

Common pitfalls.

- Trusting the callback payload instead of re-reading state.
- Retrying POST after a timeout without querying first, creating duplicates.
- Letting the model choose the assignment group.
- Approval flows with no due date.

Knowledge check.

1. How do you make ticket creation idempotent? Answer: Store an idempotency key on the record and query with sysparm_query before every create, including retries [10].
2. When would you prefer order_now over Table API inserts? Answer: When a catalog item exists for the system, so the catalog's own fulfillment runs and the request number is returned [1].
3. Where are individual approval responses stored? Answer: In sysapproval_approver [12].

Slides and assets to prepare.

- Ticket model diagram: request, requested items, approvals.
- Table API cheat sheet (methods, sysparm parameters, status codes).
- Flow Designer flow export for import into PDIs.
- Routing table template.

Sources for this module. 1, 2, 7, 10, 12

Module 5: Device accounts on F5 and Citrix NetScaler

Goal. Build F5 and NetScaler adapters that print the exact request in dry run, apply only after approval, and verify each account by read-back and test login.

Outline topics.

- F5 iControl REST: POST /mgmt/tm/auth/user with partition access and role, and the equivalent tmsh create auth user command for review
- Citrix NetScaler NITRO API: /nitro/v1/config/systemuser and binding a systemcmdpolicy for least privilege
- Dry run first: the adapter prints the exact request body and tmsh line, then applies only after approval
- Verifying the account with a read-back and a test login, and recording the result

Learning objectives.

- Learners can create an F5 local user with explicit partition access and role through iControl REST and show the equivalent tmsh command.
- Learners can create a NetScaler system user and bind a command policy through NITRO.
- Learners can implement dry-run, gated apply and verification as one adapter contract.
- Learners can handle credentials in adapters without exposing them to the model or logs.

F5 iControl REST and tmsh

- POST to /mgmt/tm/auth/user creates a user; user resources support GET, PUT, PATCH and DELETE [13].
- Set partition-access role explicitly (operator, guest, auditor and so on); the default is admin [13] [14].
- Set shell to none or tmsh; bash is only available to Administrators [14].

- Show the reviewer the tmsh equivalent: create auth user a.sharma partition-access add { Common { role operator } } shell none, with the password parameter masked [14].
- The service account the adapter uses needs the User Manager role, not Administrator, since only Administrator or User Manager can create users [14].

NetScaler NITRO

- Log in, then POST /nitro/v1/config/systemuser with username and password; success is 201 [16].
- Bind least privilege with PUT /nitro/v1/config/systemuser_systemcmdpolicy_binding {username, policyname: operator, priority} [17].
- Without a binding the user is under default DENY-ALL; the adapter treats a missing binding as a failed provision [18].
- The CLI equivalent for reviewers: add system user a.sharma, then bind system user a.sharma -policyName operator 100 [18].
- Errors: 4xx HTTP errors, 5xx NetScaler errors with details in the payload; log the error code, not the request body [16].

Dry run first

- Every adapter implements plan(item) → rendered request and CLI line, apply(item, approval_ref) and verify(item).
- Dry run prints method, URL and body with the password field replaced by a placeholder.
- apply refuses to run without an approval reference that the orchestrator re-checked in ServiceNow.
- Credentials are generated in code with the secrets module at apply time and never pass through the model [36][7].

Verification

- Read-back: GET the user and compare role, partition or bound policy with the plan [13][17].
- Test login with the new credential from the adapter host, then discard it from memory.
- Record result per device: read-back ok, login ok, or a specific failure.
- Log events with who, what, when and where; never log the password [37].

Instructor demo.

1. Run the F5 adapter in dry-run for lb-f5-01 and show the REST body and tmsh line side by side [13][14].
2. Remove the role from the body and show the dry-run linter failing because the default would be admin.
3. Approve the RITM in the PDI, apply, and show the read-back and test login.
4. Run the NetScaler adapter: create the user, bind operator, read back the binding [16][17].
5. Skip the binding on purpose and show the verification failing on a DENY-ALL user [18].

Gemnoor Tech TRANSFORMATIVE INNOVATE BUILD For more info - https://gemnoor.com

Hands-on lab. Build the F5 and NetScaler adapters, run them in dry-run mode against lb-f5-01 and lb-ns-01 in the lab, then apply with approval and verify the accounts.

1. Create adapter service accounts: User Manager on F5, a scoped admin on NetScaler; store their credentials in the lab secret store.
2. Implement the F5 adapter (plan, apply, verify) with iControl REST and a tmsh rendering.
3. Implement the NetScaler adapter with login, systemuser create, command policy binding and read-back.
4. Add a linter that fails any plan without an explicit role or policy.
5. Generate passwords with secrets and keep them out of logs and model context [36].
6. Run dry-run on lb-f5-01 and lb-ns-01 and review the output with a partner.
7. Apply with the approved RITM reference, then verify by read-back and test login.
8. Delete the test accounts at the end of the lab.

Lab success criteria.

- Dry run shows exact request bodies with masked passwords.
- Apply refuses to run without a valid approval reference.
- Both accounts read back with the planned role or policy and pass a test login.
- No password appears in logs, tickets or prompts.

Deliverable. F5 and NetScaler provisioning adapters with dry-run and verification.

Common pitfalls.

- Omitting the F5 role and getting admin.
- Creating the NetScaler user and forgetting the command policy binding.
- Using the nsroot or admin account for automation.
- Logging full request bodies, including passwords.

Knowledge check.

1. Which F5 roles can create user accounts? Answer: Administrator or User Manager [14].
2. What are the two NITRO calls for a least-privilege NetScaler user? Answer: POST /nitro/v1/config/systemuser, then PUT /nitro/v1/config/systemuser_systemcmdpolicy_binding with the policy name [16][17].
3. Why test login after read-back? Answer: Read-back proves configuration; login proves the credential and access path actually work.

Slides and assets to prepare.

- Adapter contract diagram (plan, apply, verify).
- F5 REST and tmsh side-by-side reference.
- NetScaler NITRO sequence (login, create, bind, read-back).
- Built-in NetScaler command policy table.

Sources for this module. 7, 13, 14, 16, 17, 18, 36, 37

Genmoor Tech
For more info - https://genmoor.com

Module 6: Radware, Cisco ACI and other load balancers (Anaya LB)

Goal. Add APIC and Radware adapters, define the adapter interface for other load balancers, and fall back to a precise manual ticket where no supported API path exists.

Outline topics.

- Radware Alteon and Cyber Controller: local user and role creation, and where the API stops and a ticketed manual step begins
- Cisco APIC REST API: creating aaaUser objects with aaaUserDomain and aaaUserRole for the right security domain
- An adapter interface so other load balancers (Anaya LB) plug in without changing the orchestrator
- Falling back to a ticket with exact instructions when a platform has no supported API path

Learning objectives.

- Learners can authenticate to APIC and create a local user with a security domain and role.
- Learners can explain the limits of the Radware automation path in their environment and where a manual step begins.
- Learners can implement an adapter interface that new platforms plug into without orchestrator changes.
- Learners can generate a manual ticket with exact, reviewable instructions when no adapter exists.

Cisco APIC users

- Authenticate with POST /api/aaaLogin.json; use the token as the APIC-cookie cookie; sessions time out after 600 seconds by default, so refresh with aaaRefresh or log in per run [21].
- Create the user with aaaUser > aaaUserDomain > aaaUserRole posted to /api/node/mo/uni/userext/user-<name>.json, privType writePriv or readPriv [20].
- A user with a role but no security domain has no access rights; domain assignment is mandatory [20].
- Role objects live at userext/user-X/userdomain-Y/role-Z, which is also the read-back path [24].
- Passwords cannot be read back from APIC, so a re-run that resends the password always shows a change; verify by role and domain plus a test login [23].

Radware Alteon and Cyber Controller

- Alteon has a documented REST API (the Alteon REST API User Guide is part of the documentation set) [25].
- Radware's alteon-sdk talks to Alteon over a REST backend using beans generated from the Alteon MIB; listed minimum versions are 31.0.9.0 and 32.2.1.0 [26].
- The CLI path /cfg/sys/access/user/uid <n> with name, password, cos, enable and apply is the manual fallback; a third-party guide notes a limit of 10 local users [27].

- The exact REST object for local users and the available class-of-service names must be confirmed against the lab device's own REST documentation before the course; this research did not find a public reference.
- Cyber Controller user management was not covered by any public source found; treat it as a manual-ticket path unless the customer's version documents an API.

The adapter interface

- Interface: capabilities(), plan(item), apply(item, approval_ref), verify(item), remove(item), list_users().
- The orchestrator selects adapters by platform key from the inventory; Anaya LB plugs in by adding a class, not by changing orchestration code.
- Each adapter declares which roles it can grant; the orchestrator rejects items outside that list.
- Adapters use their own scoped service credentials, never a shared admin [46].

Manual fallback

- If capabilities() lacks create_user, raise a RITM to the fulfillment group with exact steps, the target role and the verification check.
- Manual steps never include the password; the joiner sets it through the one-time channel.
- The orchestrator waits for the RITM to close and then runs a read-only verification if one exists.
- The promo's 'anaya-lb-01 no API adapter → manual RITM' is this path.

Instructor demo.

1. Log in to the DevNet ACI simulator with aaaLogin and show the APIC-cookie [21][22].
2. Dry-run and then post the a.sharma user with domain netops and role ops; read back the role subtree [20][24].
3. Post a user with a role and no domain and show the access failure [20].
4. Run the Radware adapter against the lab Alteon VA in dry-run and show the request; show the CLI fallback text [27].
5. Add an inventory entry for anaya-lb-01 with no adapter and show the manual RITM raised with exact steps.

Hands-on lab. Implement the APIC adapter against the ACI sandbox and the Radware adapter against the lab controller, and prove the orchestrator raises a manual ticket for an unsupported platform.

1. Reserve or connect to the ACI simulator sandbox [22].
2. Implement aaaLogin, session refresh and logout [21].
3. Implement the APIC adapter create, read-back and remove for aaaUser with domain and role [20][24].
4. Implement the Radware adapter using the endpoint confirmed by the instructor for the lab Alteon version, with dry-run output.
5. Define the adapter interface and refactor F5 and NetScaler adapters to it.

6. Add an unsupported platform to the inventory and implement the manual RITM fallback.
7. Run the full device plan for E10422 and record results.

Lab success criteria.

- APIC user exists with the planned domain and role and passes test login.
- The orchestrator code is unchanged when the Radware adapter is added.
- The unsupported platform produces a manual RITM with exact steps and no password.
- All adapters implement the same interface and pass the shared contract tests.

Deliverable. Multi-platform adapter set with graceful manual fallback.

Common pitfalls.

- Creating APIC users without a security domain.
- Treating APIC password re-sends as drift on every run.
- Guessing a Radware REST path from another vendor's API.
- Writing passwords into manual ticket instructions.

Knowledge check.

1. What three objects make an APIC local user usable? Answer: aaaUser, aaaUserDomain and aaaUserRole with a privilege type [20].
2. Why does an APIC adapter that resends the password always report a change? Answer: The password is a secure property that cannot be read back from APIC [23].
3. What does the orchestrator do for a platform with no create_user capability? Answer: Raises a manual RITM with exact steps and verification, and waits for it to close.

Slides and assets to prepare.

- APIC user object tree diagram.
- Adapter interface and capability table.
- Manual ticket template.
- Radware lab notes for the specific Alteon version used (instructor prepared).

Sources for this module. 20, 21, 22, 23, 24, 25, 26, 27, 46

Module 7: Repository and infrastructure access as code with GitHub and Terraform

Goal. Express GitHub team and repository access in Terraform, have Claude Code open the pull request, plan in GitHub Actions and apply only after review.

Outline topics.

- GitHub REST API for organisation invitations and team membership, and why access should live in Terraform instead

- Terraform github provider: github_team_membership and github_repository_collaborator, with terraform plan, apply and state as the record
- Claude Code opens the pull request with the plan output in the description; GitHub Actions runs plan on the PR and apply on merge
- Protecting the Terraform state and the GitHub token used by Actions

Learning objectives.

- Learners can explain the GitHub REST calls for invitations and team membership and why Terraform is the system of record.
- Learners can write github_team_membership and github_repository_collaborator resources correctly.
- Learners can build a workflow that plans on pull requests and applies after merge behind an environment approval.
- Learners can protect Terraform state and the GitHub token used by Actions.

GitHub REST and why not to use it directly

- Organization invitation: POST /orgs/{org}/invitations with invitee_id or email, role direct_member [29].
- Team membership: PUT /orgs/{org}/teams/{team_slug}/memberships/{username}, role member or maintainer; a non-member gets an invitation and pending state [28].
- Direct API changes leave no reviewable history of who granted what; Terraform state and pull requests do.
- Invitation creation is rate limited (50 per 24 hours for some organizations), which matters for bulk onboarding [29].

Terraform github provider resources

- github_team_membership { team_id = 'netops-lb', username = 'a-sharma', role = 'member' } [30].
- Do not mix github_team_membership with github_team_members on the same team [30].
- The user is not part of the team until they accept the organization invitation; plan for a pending state [30].
- Prefer team access; use github_repository_collaborator (permission pull, triage, push, maintain or admin; default push) only for exceptions, and always set permission explicitly [31].

Pull request, plan, apply

- Claude Code writes the change in access/github/*.tf and opens the PR with the plan summary in the description.
- The plan job uses hashicorp/setup-terraform@v4 and posts the plan as a PR comment; long plans go to $GITHUB_STEP_SUMMARY because comments cap at 65535 characters [33].
- The apply job runs on merge to main in an environment that requires a reviewer and prevents self-review [32].

- The promo's 'terraform plan +1 github_team_membership 0 to change 0 to destroy' is what reviewers should see for a joiner.

Protecting state and tokens

- State and plan files store secret values in plain text; use a remote backend with encryption at rest and restricted access [34].
- Put the GitHub token for the provider in the environment's secrets so it is released only after approval [32].
- Use a token with only the organization permissions needed for membership management.
- Use ephemeral values or write-only arguments where providers support them to keep secrets out of state (Terraform 1.11+) [34].

Instructor demo.

1. Show the GitHub REST call for team membership against the throwaway organization, then revert it [28].
2. Ask Claude Code to add a.sharma to netops-lb in Terraform and open the PR.
3. Show the Actions plan comment with one resource to add [33].
4. Approve the PR, merge, and show the apply job waiting for environment approval [32].
5. Show the applied state and the pending invitation behavior [30].

Hands-on lab. Have Claude Code draft the Terraform change for the joiner, open the pull request, review the Actions plan output and merge to apply.

1. Fork the access-as-code repository into the course organization.
2. Configure the remote backend and the environment with required reviewers and prevent self-review [32].
3. Store the provider token as an environment secret.
4. Ask Claude Code to add the joiner's team membership based on access_plan.json.
5. Review the diff yourself before Claude Code opens the PR.
6. Check the plan comment for exactly one addition [33].
7. Have your partner approve and merge; approve the apply environment.
8. Confirm membership (or pending invitation) in GitHub and the new state version.

Lab success criteria.

- The change is in Terraform, not made by direct API call.
- The plan shows only the expected resource change.
- Apply runs only after merge and environment approval by someone other than the author.
- State is in the remote backend and the token is an environment secret.

Deliverable. Access-as-code pipeline from PR to applied state.

Common pitfalls.

- Mixing github_team_members and github_team_membership.
- Expecting immediate team membership before the joiner accepts the org invitation.
- Running apply on pull request events.
- Committing state files to the repository.

Knowledge check.

1. Why is Terraform preferred over direct REST calls for GitHub access? Answer: Changes are reviewed in pull requests and the state is the record of who has what.
2. What happens if the joiner has not accepted the organization invitation? Answer: They are not part of the team until they accept [30].
3. How do you ensure the person who wrote the change cannot also approve the apply? Answer: Use an environment with required reviewers and prevent self-review [32].

Slides and assets to prepare.

- PR-plan-apply pipeline diagram.
- Sample workflow YAML with plan and apply jobs.
- Terraform module for team membership.
- State protection checklist.

Sources for this module. 28, 29, 30, 31, 32, 33, 34

Day 3

Module 8: Guardrails, approvals and audit for credential workflows

Goal. Make it impossible for the agent to see a password, request an out-of-matrix role or act without approval, and record every step in an append-only audit log.

Outline topics.

- Credentials are generated by code and delivered through a one-time channel; the model never sees a password
- Least privilege enforced by the matrix: the agent cannot request a role the matrix does not grant
- Human approval gates on every create, modify and delete, with the approver and decision recorded
- Append-only audit log of plan, approval, API call, response and verification, kept with the ticket

Learning objectives.

- Learners can generate credentials in code and deliver them through a one-time channel.
- Learners can enforce the matrix in code and with hooks so the agent cannot request roles outside it.
- Learners can implement approval gates for create, modify and delete with approver and decision recorded.

GenNoorTech PLAN. INNOVATE. BUILD. For more info https://gennoor.com

- Learners can design an append-only audit log that meets OWASP logging guidance.

The model never sees a password

- Generate passwords with secrets.choice over a defined alphabet, looping until policy rules are met; use token_urlsafe(32) for one-time tokens [36].
- Deliver via Vault response wrapping: write the credential, return a single-use wrapping token with X-Vault-Wrap-TTL (for example 24h) and send only the unwrap link [35].
- If the joiner's unwrap fails or the token was already used, treat it as a security alert and rotate [35].
- Credentials are sensitive information OWASP warns can leak through LLM output; redact before any data reaches the model [7].

Least privilege enforced by code

- The resolver is the only source of entitlements; plan items without a matrix rule id are rejected.
- A PreToolUse hook on the planner tools rejects any plan_account input naming an admin-class role, with exit code 2 and a reason [43].
- Deny rules apply even in bypassPermissions, and hooks run first, so the guardrail cannot be talked around [40].
- OWASP: authorization in downstream systems, not in the LLM [46]; the device adapters re-check the matrix before apply.

Approval gates

- Every create, modify and delete needs an approval reference re-read from ServiceNow at apply time.
- Record approver, decision, timestamp and the approval record id from sysapproval_approver [12].
- AC-2(e) requires approval for account creation; AC-2(f) covers create, enable, modify, disable and remove [38].
- Approval expiry: an approval older than the plan version or past its due date is invalid.

Append-only audit log

- Events: plan created, rejection, ticket created, approval, API call, response code, verification, hand-over, revocation.
- Each event records when, where, who and what, and excludes passwords, tokens and session ids [37].
- Write to an append-only store, hash-chain entries for tamper detection, and copy to read-only storage quickly [37].
- Attach the audit trail to the ServiceNow request so auditors find it with the ticket.

Instructor demo.

1. Ask the agent to give the joiner f5:admin 'because the manager asked'; show the resolver rejection and the hook block [43].
2. Show the plan with the rejection recorded and the audit event.
3. Generate a credential, wrap it in Vault and show the unwrap link; unwrap once, then show the second unwrap failing [35].
4. Grep the logs, prompts and ticket for the password and show zero hits.
5. Trace E10422 through the audit log from plan to verification, then tamper with one line and show the hash-chain check failing.

Hands-on lab. Attempt to make the agent grant an administrator role outside the matrix, watch the guardrail refuse, and trace one complete joiner through the audit log.

1. Implement credential generation with secrets and the policy check loop [36].
2. Implement Vault response wrapping delivery and the unwrap check [35].
3. Write the PreToolUse hook that blocks admin-class roles and register it in .claude/settings.json [43].
4. Add matrix re-checks inside each device adapter before apply.
5. Implement the approval re-read and expiry rules.
6. Implement the hash-chained append-only audit log with redaction [37].
7. Try three bypass prompts (urgency, manager instruction, 'temporary admin') and record the refusals.
8. Trace one joiner end to end and export the audit trail to the request.

Lab success criteria.

- All three bypass attempts are refused by code, with audit events.
- No password or token appears in any log, ticket, prompt or model output.
- Every device action references a valid, re-checked approval.
- The audit chain verifies, and a tampered entry is detected.

Deliverable. Guardrail policy, approval gate and audit log.

Common pitfalls.

- Relying on the system prompt to stop admin requests.
- Emailing passwords or putting them in ticket comments.
- Logging full API responses that contain session tokens.
- Approvals that never expire.

Knowledge check.

1. Why is response wrapping suitable for one-time delivery? Answer: The wrapping token can be unwrapped only once, so only one party sees the secret, and a failed unwrap signals interception [35].
2. Where is the 'no admin roles' rule enforced? Answer: In the resolver, a PreToolUse hook and the device adapters; not in the prompt [43][46].

GenNoorTech
For more info - https://gennoor.com

3. What must audit entries never contain? Answer: Passwords, access tokens, session identifiers or keys [37].

Slides and assets to prepare.

- Guardrail layers diagram (resolver, hook, adapter re-check, approval re-read).
- Credential delivery sequence with Vault wrapping.
- Audit event schema.
- Bypass prompt set for the lab.

Sources for this module. 7, 12, 35, 36, 37, 38, 40, 43, 46

Module 9: Verification, hand-over and reversal for leavers and movers

Goal. Verify the joiner on every platform, send the hand-over, run the workflow in reverse for leavers and movers, and detect drift with periodic access review.

Outline topics.

- Post-provisioning verification: read-back on every platform, test authentication, ticket closure in ServiceNow
- Hand-over message to the joiner and manager drafted by Claude from the verified results
- Running the workflow in reverse: disable and remove accounts, revoke GitHub teams, record the leaver
- Periodic access review that compares live accounts with the matrix and raises tickets for drift

Learning objectives.

- Learners can run post-provisioning verification and close ServiceNow items with results.
- Learners can have Claude draft the hand-over message only from verified results.
- Learners can run the leaver workflow that disables and removes accounts and revokes GitHub access.
- Learners can run an access review that compares live accounts with the matrix and raises tickets for drift.

End-to-end verification

- Read-back on every platform: GET /mgmt/tm/auth/user/<name> [13], systemuser and its binding [16][17], the APIC user and role subtree [24], GitHub team membership [28].
- Test login per platform, recorded as pass or fail.
- Close each requested item with PATCH /api/now/table/sc_req_item/{sys_id} setting state and work notes [10].
- Partial success leaves the request open with failed items reassigned.

Hand-over drafted from verified facts

GenNoorTech TRAINING INNOVATE BUILD For more info: https://gennoor.com

- Claude drafts the message to joiner and manager from a structured verification result, not from the plan.
- Code checks the draft mentions only verified systems and includes the one-time unwrap link, not a password [35].
- Include what the joiner must do: accept the GitHub invitation [30] and set their password via the link.
- Send through the organization's normal channel and log the send event [37].

Leavers and movers

- AC-2 requires notifying account managers when users are terminated or transferred and aligning account management with those processes [38].
- Leaver order: disable first where supported (APIC accountStatus inactive [23]), then delete: DELETE /mgmt/tm/auth/user/<name> on F5 [13], DELETE /nitro/v1/config/systemuser/<name> on NetScaler [16].
- GitHub: remove the Terraform resources by PR; for urgent cases DELETE /orgs/{org}/memberships/{username} removes the member or cancels a pending invitation [29].
- Dynamic secrets issued through Vault are revoked by lease or prefix, for example vault lease revoke -prefix [5].
- Movers: compute the diff between old and new role; remove what the new role does not grant before adding; for Entra groups use POST /groups/{id}/members/$ref with retry on replication delay [4].

Periodic access review

- Read live users from every platform and compare with the matrix and the joiner register.
- Drift types: unknown account, over-privileged account, account for a leaver, expired entitlement.
- Raise one ticket per drift item to the owner; never auto-delete during review.
- AC-2(j) requires reviewing accounts at a defined frequency [38].

Instructor demo.

1. Run verify for E10422 and show the per-platform table and closed RITMs.
2. Generate the hand-over draft and show the check rejecting a version that mentions an unverified system.
3. Run the leaver workflow and show each removal call and read-back confirming absence.
4. Open the Terraform PR removing the team membership and apply it [30].
5. Run access review on a lab where one stale admin account exists and show the drift ticket.

Hands-on lab. Verify the joiner end to end, send the hand-over, then run the leaver workflow and prove every account and team membership is gone.

1. Implement verify() aggregation and RITM closure through the Table API [10].
2. Implement the hand-over drafting with structured input and the fact check.

3. Implement remove() for F5, NetScaler, APIC and Radware (or manual RITM) adapters.
4. Implement the GitHub leaver path via Terraform PR.
5. Run the leaver workflow for E10422 with approval.
6. Prove absence with read-backs on every platform and GitHub.
7. Implement the access review job and run it against the lab with seeded drift.

Lab success criteria.

- Every platform shows the account absent after the leaver run.
- The GitHub membership is removed and state updated.
- The hand-over mentions only verified systems and no password.
- Access review raises a ticket for each seeded drift item.

Deliverable. Verification, hand-over and offboarding workflow.

Common pitfalls.

- Deleting before disabling where audit needs the account record.
- Forgetting pending GitHub invitations for leavers who never accepted.
- Auto-deleting drift found during review.
- Drafting hand-over messages from the plan instead of verified results.

Knowledge check.

1. How do you remove a GitHub leaver who never accepted the invitation? Answer: DELETE /orgs/{org}/memberships/{username} also cancels a pending invitation [29]; in the Terraform flow, remove the resource and apply.
2. Why should access review raise tickets rather than delete accounts? Answer: Drift may be legitimate or need investigation; deletion is a consequential action that needs approval.
3. Which AC-2 item ties leavers to account management? Answer: Notification on termination or transfer (h) and alignment with termination and transfer processes (l) [38].

Slides and assets to prepare.

- Verification matrix template.
- Leaver sequence per platform.
- Hand-over message template.
- Access review report sample.

Sources for this module. 4, 5, 10, 13, 16, 17, 23, 24, 28, 29, 30, 35, 37, 38

Module 10: Capstone: one joiner, every system, fully audited

Goal. Run one joiner through every system with approval and audit, reverse it as a leaver, and hand over a runbook for operating the workflow.

Outline topics.

Gennoor Tech
LEARN · INNOVATE · BUILD
for more info: https://gennoor.com

- Joiner record in, tickets raised, accounts provisioned, repository access applied, hand-over sent, with no credential created without approval
- Run the leaver reversal on the same joiner and show the audit log closes every entry
- Operating the workflow after the course: matrix ownership, adapter versioning and periodic access review
- Acceptance criteria review, limitations statement, demo to the group

Learning objectives.

- Learners can demonstrate the full joiner workflow with no credential created without approval.
- Learners can run the leaver reversal and show the audit log closes every entry.
- Learners can document matrix ownership, adapter versioning and access review cadence.
- Learners can state the limitations of their workflow.

Capstone flow

- Joiner record in, plan, tickets, approval, device accounts, Terraform PR, verification, hand-over.
- Every credential creation has an approval reference re-read at apply time [38].
- Every step appears in the audit log with no secret values [37].
- Leaver reversal on the same joiner closes each open entry.

Operating after the course

- Matrix ownership: each row has an owner who approves changes by pull request.
- Adapter versioning: pin device firmware versions tested per adapter; F5 trial and VPX Express builds used in the lab may differ from production [15][19].
- Access review cadence and drift ticket handling [38].
- Keep MCP servers and Claude Code settings under code review [41][42].

Acceptance and limitations

- Acceptance: tickets raised once, accounts least-privilege, approvals enforced, verification recorded, leaver complete, audit verifiable.
- Limitations: platforms without API paths, password non-idempotency on APIC [23], invitation acceptance delays on GitHub [30], PDI lifecycle in the lab [11].
- State what the agent does and does not decide.
- Demo to the group with one deliberate guardrail refusal.

Instructor demo.

1. Show the capstone acceptance checklist.
2. Run a full joiner and leaver in under fifteen minutes on the reference solution.
3. Show the audit log summary: plan, approval, device calls, verification, refusals, secrets seen by model: 0.
4. Show a sample runbook and limitations page.

GenNoor Tech
For more info - https://gennoor.com

Hands-on lab. Capstone build and demo.

1. Receive a new joiner record with a different role and one out-of-matrix request.
2. Update the matrix by pull request if the role needs a new row, with owner approval.
3. Run the full joiner workflow with approvals.
4. Verify on every platform and send the hand-over.
5. Run the leaver workflow on the same joiner and prove absence.
6. Export and verify the audit chain.
7. Write the operating runbook and limitations statement.
8. Demo to the group, including a guardrail refusal.

Lab success criteria.

- Zero credentials created without a valid approval.
- All systems verified after joiner and absent after leaver.
- Audit chain verifies and contains no secrets.
- Runbook covers matrix ownership, adapter versioning and access review.

Deliverable. End-to-end access automation workflow and runbook.

Common pitfalls.

- Skipping the leaver run because the joiner took longer than planned.
- Demos that hide manual workarounds.
- Runbooks that do not name owners.
- Leaving lab accounts and tokens active after the course.

Knowledge check.

1. What proves no credential was created without approval? Answer: Every create event in the audit log has an approval reference re-read from ServiceNow before apply.
2. Name two limitations that should appear in every capstone statement. Answer: Platforms without supported APIs fall back to manual tickets, and some properties such as APIC passwords cannot be read back [23].
3. Who approves matrix changes? Answer: The named row owner, through a pull request.

Slides and assets to prepare.

- Capstone acceptance checklist.
- Runbook template.
- Demo script template.
- Lab teardown checklist.

Sources for this module. 11, 15, 19, 23, 30, 37, 38, 41, 42

6. Capstone plan

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info: https://gennoor.com

A repeatable workflow that takes a new-joiner record, resolves entitlements from a versioned matrix, raises the right ServiceNow tickets in the right channels, provisions least-privilege accounts on F5, NetScaler, Radware, Cisco ACI and other load balancers (Anaya LB), applies GitHub and Terraform access through a reviewed pull request, and leaves a complete audit log, with the same workflow running in reverse for leavers.

A repeatable workflow that takes a new-joiner record, resolves entitlements from a versioned matrix, raises the right ServiceNow tickets, provisions least-privilege accounts on F5, NetScaler, Radware, Cisco ACI and other load balancers (Anaya LB) or raises manual tickets, applies GitHub access through a reviewed Terraform pull request, delivers credentials through a one-time channel and leaves a complete audit log, with the same workflow running in reverse for leavers.

Milestones

1. Matrix and schema validated, with owners.
2. Agent returns a validated access plan with rejections.
3. Tickets raised idempotently and gated by approval.
4. Device accounts created, verified and removable on every lab platform or via manual ticket.
5. GitHub access applied through PR, plan and gated apply.
6. Guardrails, delivery and audit chain verified; leaver run complete; runbook written.

Evaluation notes

- Automatic fail if any credential is created without a valid approval, or any secret appears in logs, prompts or tickets.
- Score least privilege by checking every live account against the matrix after the joiner run.
- Score completeness by read-back absence on every platform after the leaver run.
- Score idempotency by running the joiner twice and counting duplicates (target zero).
- Runbook and limitations statement are required for a pass.

Assessment weights

Genroor Tech
For more info: https://genroor.com

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. 'New joiner Monday, still no access Thursday' with five tickets in five queues is a teaching scenario, not a measured statistic. It is realistic because each system has a separate owner and role model (F5 roles [14], NetScaler command policies [18], APIC domains and roles [20], GitHub team roles [28]) and AC-2 expects approval before account creation [38], so manual hand-offs multiply.

GenNoorTech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

Claim or on-screen text	Basis
'7 systems, one record: ServiceNow, F5, NetScaler, Radware, Cisco ACI, GitHub and Terraform.'	Course scope from the client use-case list. Terraform is the mechanism for GitHub access rather than a separate account target; Anaya LB is handled by adapter or manual ticket.
'10 build steps' / 'ten increments across three days'.	Matches the ten modules in the outline.
'0 credentials created without approval.'	Course acceptance criterion enforced by the approval re-read in adapters; it is a design target for the lab, not an industry statistic. Aligns with AC-2(e) [38].
'matrix v1.4: lb-operator → f5:Common/operator ns:operator apic:netops/ops'.	Illustrative fixture. operator is a real F5 role [14] and a built-in NetScaler command policy [18]; ops is among APIC role names shown in Cisco's example [20]; 'netops' is a fictional security domain.
'rejected: f5:admin not granted to lb-operator (matrix rule R-12)'.	Lab outcome of the resolver; rule id is fictional.
'POST /api/now/table/sc_request requested_for=E10422' and one RITM per system routed to groups.	Table API POST creates one record per call [10]; group names are fixtures. Flow 'joiner-access-v3' waiting on the manager reflects Ask for Approval [12].
'POST /mgmt/tm/auth/user a.sharma role=operator' and 'tmsh create auth user a.sharma partition-access ...'.	Supported by F5 iControl REST and tmsh references [13][14].
'POST /nitro/v1/config/systemuser policy=operator'.	User creation is POST /nitro/v1/config/systemuser [16]; the policy is bound with a separate systemuser_systemcmdpolicy_binding call [17]. The panel compresses two calls into one line.
'lb-rdw-01 Alteon REST: create local user a.sharma role=operator'.	Alteon has a documented REST API [25][26], but the exact user object and whether an 'operator' class of service exists were not verified from a public source. Treat as illustrative until confirmed on the lab build.
'POST /api/node/mo/uni/userext/user-a.sharma.json'.	Matches Cisco's documented JSON URL pattern for local users [20].
'anaya-lb-01 no API adapter → manual RITM'.	Course design; no public API documentation for Anaya LB was found.
'verify: 5/5 read-back ok test-login 5/5 ok credentials sent via one-time link'.	Illustrative output; one-time delivery uses Vault response wrapping [35].
'actions: terraform plan +1 github_team_membership ... merged ... apply: ok state: v87'.	Illustrative; resource and plan-on-PR flow per [30][33]; apply gated by environment reviewers [32].

GenoTech TRAINING INNOVATION For more info: https://genoor.com

'secrets seen by model: 0' and 'Passwords are generated by code and delivered through a one-time channel the model never sees.'	Course guardrail design; password generation with Python secrets [36], redaction per OWASP LLM02 [7]. The count is a lab audit metric, not a guarantee for other deployments.
'leaver drill: 5/5 accounts removed 1 team revoked RITM closed'.	Illustrative output of the leaver lab using documented delete calls [13][16][29].
'Every lab runs against a sandbox: a ServiceNow developer instance, virtual F5 and NetScaler appliances, a lab Radware controller, the Cisco ACI simulator and a throwaway GitHub organisation.'	PDI [11], BIG-IP VE trial [15], VPX Express [19], DevNet ACI simulator [22]. The Radware controller depends on the client supplying a lab license; otherwise a mock is used.

References

[1] Service Catalog API. ServiceNow (Yokohama API Reference). https://www.servicenow.com/docs/r/yokohama/api-reference/rest-apis/c_ServiceCatalogAPI.html
Supports: Raising tickets in the right channel with ServiceNow. The Service Catalog API lives under /api/sn_sc/servicecatalog/; POST items/{sys_id}/order_now orders an item directly (sysparm_quantity is required, with optional variables and sysparm_requested_for) and returns the resulting request number and sys_id, while POST cart/submit_order checks out the cart and behaves differently under one-step and two-step checkout.

[2] Request (Jira Service Management Cloud REST API). Atlassian Developer. https://developer.atlassian.com/cloud/jira/service-desk/rest/api-group-request/
Supports: Ticketing alternative for teams not on ServiceNow. POST /rest/servicedeskapi/request creates a customer request; the body must include serviceDeskId, requestTypeId and requestFieldValues covering every field the request type requires, needs the write:servicedesk-request scope, and returns 201 Created.

[3] Create User - Microsoft Graph v1.0. Microsoft Learn. https://learn.microsoft.com/en-us/graph/api/user-post-users?view=graph-rest-1.0
Supports: Identity source option (Microsoft Entra ID). POST /users creates an Entra ID user and requires accountEnabled, displayName, mailNickname, passwordProfile and userPrincipalName; the least-privileged permission is User.Create, with User.ReadWrite.All and Directory.ReadWrite.All as higher-privileged alternatives.

[4] Add members - Microsoft Graph v1.0. Microsoft Learn. https://learn.microsoft.com/en-us/graph/api/group-post-members?view=graph-rest-1.0
Supports: Movers: group membership changes. POST /groups/{group-id}/members/$ref adds one member with an @odata.id reference and PATCH /groups/{group-id} with members@odata.bind adds up to 20; GroupMember.ReadWrite.All is the least-privileged permission, and newly created groups may briefly return 400 due to replication delay, so callers should retry after a short wait.

[5] Lease, Renew, and Revoke | Vault. HashiCorp Developer. https://developer.hashicorp.com/vault/docs/concepts/lease
Supports: Leavers: revoking short-lived credentials. Vault attaches a lease with a TTL to every dynamic secret and service token; it revokes the secret automatically if the lease is not renewed, revoking a lease invalidates the secret immediately, revoking a token revokes all leases it created, and prefix revocation (vault lease revoke -prefix aws/) is supported.

[6] least privilege - Glossary | CSRC. NIST Computer Security Resource Center. https://csrc.nist.gov/glossary/term/least_privilege
Supports: Entitlement matrix and guardrails. NIST defines least privilege (per CNSSI 4009-2022, SP 800-53 Rev.

5 and SP 800-171r3) as restricting each user, process or entity to the minimum system resources and authorizations needed to perform its function.

[7] LLM02:2025 Sensitive Information Disclosure. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm022025-sensitive-information-disclosure/
Supports: Guardrails: keeping credentials away from the model. OWASP lists security credentials among the sensitive data an LLM application can leak, recommends input validation and redaction before data reaches the model, least-privilege access and tokenization or masking, and warns that system-prompt restrictions can be bypassed through prompt injection.

[8] Security Best Practices - Model Context Protocol. Model Context Protocol. https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices
Supports: MCP server design for the access workflow. MCP servers MUST NOT accept tokens that were not explicitly issued for the MCP server (no token passthrough), and the spec recommends a progressive least-privilege scope model with a minimal initial scope set and incremental elevation via WWW-Authenticate scope challenges, listing wildcard or omnibus scopes as a common mistake.

[9] Security - Claude Code Docs. Anthropic (Claude Code documentation). https://code.claude.com/docs/en/security
Supports: Setting up Claude Code for a credential workflow. Anthropic does not security-audit or manage any MCP server and encourages writing your own or using servers from trusted providers with per-server permissions; it recommends reviewing suggested commands before approval and not piping untrusted content directly to Claude, and stores API keys in the macOS Keychain or a 0600-mode file on Linux.

[10] Table API. ServiceNow (Yokohama API Reference). https://www.servicenow.com/docs/r/yokohama/api-reference/rest-apis/c_TableAPI.html
Supports: Raising tickets in the right channel with ServiceNow. The Table API at /api/now/table/{tableName} supports GET, POST (one record per call, 201 on success with a Location header), PUT, PATCH and DELETE (204) on /{sys_id}; sysparm_query takes encoded queries with ^ for AND, sysparm_fields limits returned fields, sysparm_limit defaults to 10000 and is applied before ACL evaluation, and the calling user needs roles that allow access to the table.

[11] Understanding PDIs. ServiceNow Developer Program documentation. https://www.servicenow.com/docs/r/SmYvGPN0OygSQs53fsexIQ/yG_RdLKb649OcfnDiGEkwQ
Supports: Lab environment: ServiceNow instance. Personal Developer Instances are ServiceNow environments assigned to developers for building and testing; idle instances hibernate with data preserved and wake on sign-in, and PDIs unused for ten days are returned to the pool, reset to their original state and reassigned.

[12] Flow Designer Approvals Overview - Workflow Automation CoE. ServiceNow Community (Workflow Automation Center of Excellence, ServiceNow employee author). https://www.servicenow.com/community/workflow-automation-articles/flow-designer-approvals-overview-workflow-automation-center-of/ta-p/2528202
Supports: Flow Designer manager approval. The Ask for Approval action supports rules such as anyone approves, all approve, a set number or a percentage, combined with AND/OR; it returns an approval state (approved, rejected, cancelled) that should be followed by If/Else logic; individual responses are stored in sysapproval_approver; a due date can auto-approve, reject or cancel pending approvals.

[13] APIRef_tm_auth_user. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_auth_user.html
Supports: Device accounts on F5. The iControl REST collection /mgmt/tm/auth/user supports GET, and user resources support GET, PUT, PATCH, DELETE and POST, keyed by name and partition; properties include password, description, shell (bash, tmsh or none) and a partition-access array whose role defaults to admin when not specified.

[14] auth user (tmsh reference). F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/auth/auth_user.html
Supports: Device accounts on F5. tmsh 'create auth user [name]' takes partition-access add { [partition] { role [role-name] } }, password, prompt-for-password, shell and session-limit; roles include admin, operator, guest,

auditor, user-manager, resource-admin, manager, certificate-manager, irule-manager and others; bash shell is only for Administrators; only Administrator or User Manager roles can create or modify users. Example: create user tknox password aBcD007 partition-access add { partition1 { role operator } }.

[15] Try BIG-IP and BIG-IQ Virtual Editions. F5. https://www.f5.com/trials/big-ip-virtual-edition
Supports: Lab environment: F5 BIG-IP VE. F5 offers a 30-day free trial of BIG-IP VE and BIG-IQ VE requested through MyF5 (My Products & Plans > Trials); BIG-IP VE 12.1.x and later runs on VMware ESXi, KVM, Xen, Hyper-V and major clouds; the trial BIG-IP VE is limited to 1 Mbps throughput and needs at least 10 GB disk and 2 GB memory per vCPU.

[16] systemuser | NITRO API configuration. NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/system/systemuser.html
Supports: Device accounts on NetScaler. A system user is created with POST http://<netscaler-ip>/nitro/v1/config/systemuser with mandatory username and password and optional externalauth, promptstring, timeout, logging, maxsession and allowedmanagementinterface; success returns 201 Created; DELETE /nitro/v1/config/systemuser/<username> returns 200; 4xx are HTTP errors and 5xx NetScaler-specific errors with details in the payload.

[17] systemuser_systemcmdpolicy_binding | NITRO API configuration. NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/system/systemuser_systemcmdpolicy_binding.html
Supports: Device accounts on NetScaler. A command policy is bound to a user with PUT /nitro/v1/config/systemuser_systemcmdpolicy_binding with username, policyname and priority, sending the NITRO auth token as a cookie; the binding is removed with DELETE /nitro/v1/config/systemuser_systemcmdpolicy_binding/<username>?args=policyname:<name>.

[18] User, user groups, and command policies. NetScaler documentation. https://docs.netscaler.com/us/en/citrix-adc/current-release/system/authentication-and-authorization-for-system-user/user-usergroups-command-policies.html
Supports: Least privilege on NetScaler. Built-in command policies are read-only, operator (read-only plus enable/disable of services and servers), network, sysadmin and superuser (same privileges as nsroot); users or groups with no bound policy fall under a default DENY-ALL and cannot run configuration commands; CLI equivalents are 'add system user <name>' and 'bind system user <userName> -policyName <policyName> <priority>'.

[19] NetScaler VPX FAQ (13.1). NetScaler documentation. https://docs.netscaler.com/en-us/vpx/13-1/deploy-vpx-faq
Supports: Lab environment: NetScaler VPX. VPX Express is free, requires no license file and comes with no commitment; it offers the Standard Edition feature set except Gateway; a customer can download at most five VPX Express instances; Express users cannot file technical support cases but can use the knowledge center and forums.

[20] Cisco APIC REST API Configuration Guide, Release 4.0(x) - Managing Roles, Users, and Signature-Based Transactions. Cisco. https://www.cisco.com/c/en/us/td/docs/switches/datacenter/aci/apic/sw/4-x/rest-api-config/Cisco-APIC-REST-API-Configuration-Guide-401/Cisco-APIC-REST-API-Configuration-Guide-401_chapter_010.html
Supports: Cisco ACI local users. A local user is created by posting an aaaUser (name, pwd) with a child aaaUserDomain (e.g. name=all) containing aaaUserRole (name, privType writePriv or readPriv) to https://<apic>/api/node/mo/uni/userext.xml, or as JSON to /api/node/mo/uni/userext/user-<name>.json; creating a user and assigning a role does not enable access rights unless the user is also assigned to security domains; the fabric has pre-created domains all and infra.

[21] Cisco APIC REST API Configuration Guide, Release 4.2(x) and Later - Using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-rest-api-configuration-guide/cisco-apic-rest-api-configuration-guide-42x-and-later/m_using_the_rest_api.html
Supports: Cisco ACI adapter authentication. Clients log in with POST https://<apic>/api/aaaLogin.json carrying an

aaaUser with name and password; the returned token is used as a cookie named APIC-cookie; the default session timeout is 600 seconds; aaaRefresh renews the session and returns a new token; aaaLogout closes it.

[22] ACI Sandboxes. Cisco DevNet. https://developer.cisco.com/docs/aci/sandbox
Supports: Lab environment: Cisco ACI. Cisco DevNet offers an always-on ACI Simulator sandbox (shared, no reservation) and a reservable ACI Simulator sandbox that can be booked for up to five consecutive days to make APIC API requests.

[23] aci_aaa_user module source (CiscoDevNet ansible-aci). Cisco DevNet (GitHub).
https://raw.githubusercontent.com/CiscoDevNet/ansible-aci/master/plugins/modules/aci_aaa_user.py
Supports: Cisco ACI adapter: idempotency and read-back. The aaaUser object is addressed as userext/user-{name}; the module maps enabled to accountStatus active/inactive and supports expiration, expires and clear_password_history; it is not idempotent when aaa_password is used because the password is a secure property that cannot be retrieved from APIC.

[24] aci_aaa_user_role module source (CiscoDevNet ansible-aci). Cisco DevNet (GitHub).
https://raw.githubusercontent.com/CiscoDevNet/ansible-aci/master/plugins/modules/aci_aaa_user_role.py
Supports: Cisco ACI adapter: role objects. aaaUserRole sits under aaaUserDomain under the user, addressed as userext/user-{user}/userdomain-{domain}/role-{role}; privilege_type read or write maps to readPriv and writePriv.

[25] Alteon Release Notes 31.0.11.0 - Related Documentation. Radware.
https://portals.radware.com/releasenotes/Alteon_Release_Notes_31.0.11.0/Alteon_Release_Notes/AlteonOS-31_0_11_0-RN.1.51.html
Supports: Radware Alteon automation surface. Radware's Alteon documentation set includes an Alteon REST API User Guide alongside the Command Line Interface Application Guide, Command Reference, Web Based Management guide and Alteon VA Installation and Maintenance Guide.

[26] alteon-sdk (PyPI). Radware (PyPI). https://pypi.org/project/alteon-sdk/0.6b38/
Supports: Radware Alteon adapter. alteon-sdk is a Python SDK for interacting with Alteon structures via a REST backend over HTTPS (AlteonClient with server, credentials, port and timeout); its beans are auto-generated from the Alteon MIB and configurators support READ, READ_ALL, DELETE, UPDATE and DEPLOY; listed minimum Alteon versions are 31.0.9.0 and 32.2.1.0 (this page is a 2019 pre-release).

[27] 2.1.10 Radware (adding a user). Indeni (third-party integration guide). https://indeni.com/docs/user-guide/part-2-getting-started-2/2-1-adding-user/radware-2/
Supports: Radware Alteon manual fallback steps. A third-party guide documents the Alteon CLI path /cfg/sys/access/user/uid <#> to create a local user, set name and password, set the privilege with 'cos', enable and apply, and notes that up to 10 local users can be defined.

[28] REST API endpoints for team members. GitHub Docs. https://docs.github.com/en/rest/teams/members
Supports: Repository access with GitHub. PUT /orgs/{org}/teams/{team_slug}/memberships/{username} adds a member with role member (default) or maintainer; if the person is not an organization member an owner can add them and GitHub emails an invitation, leaving membership pending; DELETE on the same path removes team membership without deleting the user; it returns 403 when team synchronization is set up.

[29] REST API endpoints for organization members. GitHub Docs. https://docs.github.com/en/rest/orgs/members
Supports: Repository access and leaver removal. POST /orgs/{org}/invitations invites by email or invitee_id with role direct_member (default), admin, billing_manager or reinstate and optional team_ids; DELETE /orgs/{org}/memberships/{username} (owners only) removes a member or cancels a pending invitation and returns 204; invitation creation is subject to rate limits (owners are capped at 50 invitations per 24 hours, or 500 for organizations older than a month or on paid plans).

[30] github_team_membership (terraform-provider-github docs). GitHub (integrations/terraform-provider-github).
https://github.com/integrations/terraform-provider-github/blob/main/docs/resources/team_membership.md
Supports: Access as code with Terraform. github_team_membership takes team_id (id or slug), username and role (member or maintainer, default member); it imports as teamid:username or teamname:username; it is not compatible with github_team_members; a user who has not accepted the organization invitation is not part of the team until they do.

[31] github_repository_collaborator (terraform-provider-github docs). GitHub (integrations/terraform-provider-github). https://github.com/integrations/terraform-provider-github/blob/main/docs/resources/repository_collaborator.md
Supports: Access as code with Terraform. github_repository_collaborator takes repository, username and permission (pull, triage, push, maintain, admin or a custom role; default push); applying sends an invitation, destroying cancels the invitation or removes the collaborator; it is non-authoritative and cannot be combined with github_repository_collaborators.

[32] Managing environments for deployment - GitHub Docs. GitHub Docs. https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments/manage-environments
Supports: Apply gate in GitHub Actions. An environment can require up to six reviewers (people or teams), where one approval lets the job proceed, can prevent users approving runs they triggered, restricts deploying branches and tags, and exposes environment secrets only after its protection rules pass.

[33] hashicorp/setup-terraform. HashiCorp (GitHub). https://github.com/hashicorp/setup-terraform
Supports: Terraform plan on pull requests. hashicorp/setup-terraform@v4 installs the Terraform CLI (terraform_version input), optionally a wrapper exposing stdout, stderr and exitcode; the documented workflow runs fmt, init, validate and plan and posts the plan as a PR comment with actions/github-script, needing pull-requests: write; GitHub comments are limited to 65535 characters, so long plans can go to $GITHUB_STEP_SUMMARY instead.

[34] Manage sensitive data in your configuration | Terraform. HashiCorp Developer. https://developer.hashicorp.com/terraform/language/state/sensitive-data
Supports: Protecting state and tokens. Secret values in configuration are stored in plain text in state and plan files; HashiCorp recommends a remote backend, encryption at rest (automatic in HCP Terraform) and restricted, audited state access; ephemeral values and write-only arguments (usually ending in _wo, Terraform 1.11+) keep values out of state and plan.

[35] Response wrapping | Vault. HashiCorp Developer. https://developer.hashicorp.com/vault/docs/concepts/response-wrapping
Supports: One-time credential delivery. With response wrapping Vault places a response in the cubbyhole of a single-use token and returns that token; the TTL is set with the X-Vault-Wrap-TTL header or -wrap-ttl flag; sys/wrapping/unwrap returns the original response once, so only one party can see it; sys/wrapping/lookup and creation-path checks help detect interception, and a failed unwrap should be treated as a security alert.

[36] secrets - Generate secure random numbers for managing secrets. Python Software Foundation. https://docs.python.org/3/library/secrets.html
Supports: Generating credentials in code. The secrets module should be used instead of random for security purposes; secrets.choice picks from a sequence and token_urlsafe(nbytes) returns URL-safe Base64 text; the docs give a recipe that loops until a password meets character-class rules and state that 32 bytes (256 bits) of randomness is believed sufficient for typical uses.

[37] Logging - OWASP Cheat Sheet Series. OWASP. https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
Supports: Audit log design. OWASP recommends logging authentication and authorization events and use of higher-risk functions such as adding or deleting users and changing privileges, recording when, where, who and what for each event; passwords, access tokens, session identifiers and keys must be removed, masked, hashed or encrypted; logs need tamper detection, early copy to read-only media and access that is restricted and itself recorded.

[38] Access Control family (NIST SP 800-53 Rev. 5 AC-2) - FedRAMP. FedRAMP (U.S. GSA), reproducing NIST SP 800-53 Rev. 5. https://www.fedramp.gov/2026/reference/controls/access-control/
Supports: Approvals, reviews and leaver alignment. AC-2 Account Management requires approvals by defined personnel for requests to create accounts (e), creating, enabling, modifying, disabling and removing accounts per policy (f), monitoring account use (g), notifying account managers within defined periods when users are terminated or transferred (h), periodic account reviews (j) and aligning account management with personnel termination and transfer processes (l).

[39] Give Claude custom tools - Claude Agent SDK. Anthropic (Claude Code documentation). https://code.claude.com/docs/en/agent-sdk/custom-tools
Supports: Agent SDK orchestrator tools. Python tools are defined with @tool(name, description, schema) and wrapped in create_sdk_mcp_server(name, version, tools); they are passed via ClaudeAgentOptions(mcp_servers=...) and pre-approved in allowed_tools as mcp__{server}__{tool}; handlers return {'content': [...]} and may set is_error; annotations such as readOnlyHint are metadata, not enforcement; the Python dict schema does not support enums, so a full JSON Schema dict is needed for them.

[40] Configure permissions - Claude Agent SDK. Anthropic (Claude Code documentation). https://code.claude.com/docs/en/agent-sdk/permissions
Supports: Permission design for the orchestrator. The SDK evaluates hooks, then deny rules, then ask rules, then the permission mode, then allow rules, then the canUseTool callback; deny rules apply even in bypassPermissions; dontAsk mode denies anything not pre-approved; allowed_tools does not constrain bypassPermissions; disallowed_tools with a bare name removes a tool from context.

[41] Connect Claude Code to tools via MCP. Anthropic (Claude Code documentation). https://code.claude.com/docs/en/mcp
Supports: MCP servers for ServiceNow and identity data. claude mcp add supports --transport http or stdio, --scope local, project (stored in .mcp.json) or user, and --env; .mcp.json supports ${VAR} and ${VAR:-default} expansion; credential variables such as ANTHROPIC_API_KEY read as empty in remote server url and headers; project servers need approval in interactive sessions but load without a prompt in claude -p and Agent SDK runs; servers that fetch external content expose you to prompt injection.

[42] Configure permissions - Claude Code Docs. Anthropic (Claude Code documentation). https://code.claude.com/docs/en/permissions
Supports: Claude Code permission settings for a credential project. Permission rules in settings files are evaluated deny, then ask, then allow, and the first match wins; Read(./.env) or Read(./secrets/**) deny rules block file reads; mcp__<server>__<tool> and mcp__<server>__* match MCP tools; Bash rules match the command text and are not a security boundary around a program; defaultMode sets the starting permission mode.

[43] Hooks reference - Claude Code Docs. Anthropic (Claude Code documentation). https://code.claude.com/docs/en/hooks
Supports: Guardrail hooks. Hooks are configured in ~/.claude/settings.json, .claude/settings.json or .claude/settings.local.json with an event (such as PreToolUse), a matcher (exact names, lists like Edit|Write, or regex such as mcp__memory__.*) and handlers; a PreToolUse hook blocks a call by exiting with code 2 or returning permissionDecision 'deny'.

[44] How Claude remembers your project - Claude Code Docs. Anthropic (Claude Code documentation). https://code.claude.com/docs/en/memory
Supports: CLAUDE.md conventions. Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md and personal ones in CLAUDE.local.md; /init creates a starting file; @path imports other files; .claude/rules/ holds topic rules; CLAUDE.md is context rather than enforced configuration, so blocking an action needs a PreToolUse hook.

[45] Structured outputs - Claude API Docs. Anthropic. https://platform.claude.com/docs/en/build-with-claude/structured-outputs
Supports: Access plan schema. output_config.format with type json_schema makes Claude return JSON matching a schema; strict: true on a tool validates tool inputs; objects need additionalProperties false; recursive schemas, numeric minimum/maximum and string length constraints are not supported; client.messages.parse() accepts a Pydantic model and returns parsed_output.

[46] LLM06:2025 Excessive Agency. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/
Supports: Guardrails and tool design. Excessive agency comes from excessive functionality, permissions or autonomy; OWASP recommends minimal, narrowly scoped tools, avoiding open-ended tools such as shell execution, minimal permissions, user approval for high-impact actions and complete mediation, meaning authorization enforced in downstream systems rather than by the LLM.

Gennoor Tech
TRAIN. INNOVATE. BUILD.
For more info - https://gennoor.com